

Sentinel AI Security Audit Report

Audit Metadata

- **Filename:** vuln.c
- **SHA-256 Checksum:** fe9e01d2075fccf4a5a02f158d070839d8f1f5140915a56916c487f15813b114
- **Verdict:** VULNERABLE
- **Risk Level:** CRITICAL
- **Total Functions Evaluated:** 2

Executive Summary

The Sentinel AI Security Audit has concluded its automated and manual review of the provided binary object `vuln.c`. Our analysis has identified a **CRITICAL** security vulnerability within the codebase. The audit utilized a Graph Attention Network (GATv2) model to analyze the Control Flow Graph (CFG) and instruction sequences, identifying a classic stack-based buffer overflow.

The identified vulnerability allows for an out-of-bounds write, which can be leveraged by an attacker to overwrite the return address on the stack, leading to Arbitrary Code Execution (ACE) or a complete system compromise. Given the nature of the flaw, immediate remediation is required before this code is deployed in a production environment.

Analysis Metrics

- **Static Analysis Engine:** GNN-based CFG Neural Auditor (GATv2)
- **Total Functions Scanned:** 2
- **Flagged Functions:** 1
- **Average Model Confidence:** 85.00%
- **Analysis Depth:** Inter-procedural Control Flow & Register Pressure Analysis

Detailed Findings

Finding 1: Function `vuln\_c\_func`

- **Model Confidence:** 85.00%
- **Identified Threat:** CWE-119 (Improper Restriction of Operations within the Bounds of a Memory Buffer) / CWE-787 (Out-of-bounds Write)

Technical Breakdown

The disassembly reveals a critical memory safety violation in **Basic Block 1**.

1. **Stack Allocation:** The function prologue allocates 48 bytes of stack space (`SUB RSP, 0x30`).
2. **Buffer Localization:** A local buffer is established at `[RBP - 0x10]`. This provides only 16 bytes of space before reaching the saved Base Pointer (RBP) and the Return Address.
3. **Vulnerable Operation:** The instruction `CALL 0x00003c70` invokes `strcpy`. The source string is passed via `RDX`

(derived from the function input), and the destination is the 16-byte stack buffer.

4. **Exploit Vector:** Because ``strcpy`` does not perform bounds checking, an input string exceeding 15 characters (plus the null terminator) will overwrite the stack frame. An attacker can precisely craft the input to overwrite the return address, redirecting execution flow to a malicious payload (e.g., a NOP slide or Return-Oriented Programming chain).

SEI CERT C Violation

- **Rule ARR30-C:** Do not form or use out-of-bounds pointers or array subscripts.
- **Rule STR31-C:** Guarantee that storage for strings has sufficient space for character data and the null terminator.

Remediation & Secure Code Fix

The use of ``strcpy`` is deprecated in secure environments. It should be replaced with ``strncpy`` (with manual null termination) or, preferably, ``strlcpy`` or bounds-checked alternatives.

Vulnerable Pattern:

```
void vuln_c_func(char *input) {
    char buffer[16];
    if (input != NULL) {
        strcpy(buffer, input); // CRITICAL VULNERABILITY
        printf("%s", buffer);
    }
}
```

Secure Remediation (C11 Standard):

```
#include <string.h>
#include <stdio.h>

#define MAX_BUF 16

void secure_func(const char *input) {
    char buffer[MAX_BUF];

    if (input == NULL) return;

    // Use strncpy and manually ensure null termination
    strncpy(buffer, input, MAX_BUF - 1);
    buffer[MAX_BUF - 1] = '\0';

    printf("%s", buffer);
}
```

General Mitigations

To harden the application against memory corruption exploits, the following platform-level security controls should be implemented during the compilation and deployment phases:

1. **Stack Canaries (Stack Smashing Protector):** Compile with ``-fstack-protector-all`` to insert guard values (canaries) before the return address. If a buffer overflow occurs, the canary is corrupted, and the program terminates before the malicious return address can be used.
2. **Address Space Layout Randomization (ASLR):** Ensure the target OS has ASLR enabled to randomize the base addresses of the stack, heap, and libraries, making ROP-based exploitation significantly more difficult.
3. **Data Execution Prevention (DEP/NX):** Mark the stack and heap as non-executable (``-z noexecstack``) to prevent the execution of injected shellcode.

4. **Control Flow Integrity (CFI):** Utilize LLVM/Clang's CFI or hardware-based features like Intel CET to ensure that the program's execution follows the intended control flow graph.
5. **Static & Dynamic Analysis (SAST/DAST):** Integrate tools like `AddressSanitizer` (ASan) during the testing phase to catch out-of-bounds accesses at runtime.

End of Report

Sentinel AI Security Division